



سیستم‌های چندرسانه‌ای پیشرفته

گزارش تکلیف اول

پوریا صامتی

۴۰۴۰۹۷۱۴

پائیز ۱۴۰۴

## فهرست مطالب

|    |                 |
|----|-----------------|
| ۳  | پاسخ سوال اول   |
| ۴  | پاسخ سوال دوم   |
| ۵  | پاسخ پرسش سوم   |
| ۶  | پاسخ پرسش چهارم |
| ۷  | پاسخ پرسش پنجم  |
| ۱۰ | پاسخ پرسش ششم   |
| ۱۲ | پاسخ پرسش هفتم  |

## پاسخ سوال اول

در ابتدا انواع تصاویر با فرمت‌های PBM، PGM و PPM را بررسی می‌کنیم.

- تصاویر سیاه‌سفید تک‌رنگ (PBM)

- تصاویر خاکستری (PGM)

- تصاویر رنگی (PPM)

هر فایل با یک هدر (Header) که به صورت اسکی است شروع می‌شود؛ این هدر به ترتیب نوع فایل، ابعاد تصویر و مقدار بیشینه شدت روشنایی در PGM و PPM را مشخص می‌کند. در تصاویر PBM از آنجا که تصویر سیاه و سفید است، نیازی به بیان شدت روشنایی نیست و در هدر آنها حداکثر شدت روشنایی وجود ندارد. فقط نوع فایل و ابعاد تصویر بیان می‌شود. بعد از هدر، داده‌های تصویر به صورت اسکی یا باینری (قرار می‌گیرند). تصویر زیر نمونه‌ای از این هدر می‌باشد:

```
P6 1282 852 255
```

با استفاده از Magic Number که عبارتند از P1-P6 نوع فایل و فرمت آن مشخص می‌شود. برای مثال P6 بیانگر فرمت PPM است که داده آن بصورت اسکی (Ascii) در آن قرار گرفته است. به غیر از شکل اسکی، داده‌ها به شکل باینری نیز در می‌توانند ذخیره شوند. مثلاً اگر عدد جادویی P4 باشد، نشان‌دهنده این است که تصویر سیاه و سفید است و به شکل باینری ذخیره شده است (به عبارتی فایل از نوع PBM باینری است).

همچنین کاراکترهای خاص دیگری مثل فاصله، با استفاده از این فرمت‌ها مشخص می‌شوند. برای مثال در تصاویر با نوع P1 بین اجزای هدر فاصله خالی می‌تواند هر تعداد باشد و خط‌هایی که با کاراکتر # شروع می‌شوند کامنت هستند و مثل فاصله خالی در نظر گرفته می‌شوند.

## پاسخ سوال دوم

تابع زیر برای خواندن تصاویر PPM با عدد جادویی p6 طراحی شده است:

```
def read_ppm(path):  
    with open(path, 'rb') as f:  
        header_line = f.readline().strip()  
        header_line = header_line.split()  
        if header_line[0] != b'P6':  
            return -1, -1, -1, -1  
  
        header_tokens = list(map(int, header_line[1:]))  
        width, height, maxval = header_tokens  
  
        img_data = f.read(width * height * 3)  
  
        img = np.frombuffer(img_data, dtype=np.uint8).reshape((height, width, 3))  
        return img, width, height, maxval
```

در نتیجه استفاده از این تابع برای خواندن تصاویر تست سوال دوم، نتایج زیر حاصل شدند:



## پاسخ پرسش سوم

در این سوال می‌خواهیم تابعی بنویسیم که میانگین مربعات خطا (MSE) را بین تصاویر (چه خاکستری و چه رنگی) محاسبه کند. روند کار به‌صورت زیر است: ابتدا تصاویر را دریافت می‌کنیم و هر یک را به یک بردار یک‌بعدی تبدیل می‌کنیم؛ سپس بین این دو بردار مقدار MSE محاسبه می‌شود.

چالش زمانی پدید می‌آید که یکی از تصاویر خاکستری و دیگری رنگی باشد، چون تصویر رنگی سه کانال (RGB) دارد و تصویر خاکستری تنها یک کانال. برای یکسان‌سازی ابعاد، کانال تک‌کاناله خاکستری را سه بار تکثیر می‌کنیم تا مانند یک تصویر RGB سه‌کاناله شود. پس از این تبدیل، هر دو تصویر بردارسازی شده و به تابع محاسبه خطا ارسال می‌شوند (همان مکانیزمی که برای دو تصویر هم‌نوع (هر دو خاکستری یا هر دو رنگی) استفاده می‌شود).

```
def my_mse(image1: np.ndarray, image2: np.ndarray):  
    if image1.shape != image2.shape:  
        return -1  
  
    img1_flat = image1.flatten()  
    img2_flat = image2.flatten()  
  
    error = img1_flat - img2_flat  
    error_square = error ** 2  
    error_square_sum = np.sum(error_square)  
    mean_error_square_sum = error_square_sum / len(img1_flat)  
    return mean_error_square_sum
```

```
def mse_gray_color(rgb_image: np.ndarray, gray_image: np.ndarray):  
    gray_3d = np.repeat(gray_image[:, :, np.newaxis], 3, axis=2)  
    return my_mse(rgb_image, gray_3d)
```

## پاسخ پرسش چهارم

برای محاسبه معیار PSNR از تابع زیر استفاده شده است. در تعیین مقدار بیشینه شدت روشنایی از سازوکاری بهره می‌بریم که امکان کار با تصاویر دارای عمق بیش از ۸ بیت (مانند تصاویر پزشکی) را فراهم می‌کند. بدین منظور، ابتدا بیشینه شدت روشنایی موجود در دو تصویر را محاسبه می‌کنیم. سپس کوچک‌ترین توان دو که بزرگ‌تر یا مساوی این مقدار باشد را می‌یابیم و مقدار یک واحد کمتر از آن را به عنوان بیشینه شدت روشنایی انتخاب می‌کنیم. در نهایت، پس از محاسبه میانگین مربعات خطا (MSE) میان دو تصویر، مقدار PSNR را با قرار دادن این مقادیر در فرمول مربوطه به دست می‌آوریم.

```
def next_power_of_two(x):
    return 1 << (x - 1).bit_length()

def my_psnr(image1: np.ndarray, image2: np.ndarray):
    if image1.shape != image2.shape:
        return -1

    max_intensity = max(image1.max(), image2.max())

    next_pow2 = next_power_of_two(int(max_intensity + 1))
    max_value = next_pow2 - 1
    max_value_sq = max_value ** 2

    mse = np.mean((image1.astype(np.float64) - image2.astype(np.float64)) ** 2)

    if mse == 0:
        return np.inf

    return 10 * np.log10(max_value_sq / mse)
```

## پاسخ پرسش پنجم

در این بخش، برای تبدیل تصویر رنگی به تصویر خاکستری از سه روش مختلف استفاده شده است. ایده اصلی در هر سه روش یکسان است: سه کانال رنگی قرمز، سبز و آبی هر کدام در ضریبی مشخص ضرب می‌شوند؛ ضرایبی که مجموع آن‌ها برابر با ۱ است. سپس خروجی سه کانال با یکدیگر جمع می‌شود تا تصویر نهایی خاکستری به دست آید. در ادامه، این سه روش را به صورت جداگانه بررسی می‌کنیم.

**روش اول) میانگین سه کانال:** در این روش به سادگی هر سه کانال را با هم جمع می‌کنیم و بر ۳ تقسیم می‌کنیم.

```
def rgb_to_gray_average(image: np.ndarray):  
    gray_img = (image[:, :, 0]/3) + (image[:, :, 1]/3) + (image[:, :, 2]/3)  
    return gray_img
```

نتایج این روش به شرح زیر است:



**روش دوم) ضرایب لومینانس:** این روش بر پایه ضرایبی است که از ویژگی‌های سیستم بینایی انسان به دست آمده‌اند. چشم ما رنگ‌ها را با حساسیت‌های متفاوتی درک می‌کند؛ برای مثال، سبز بسیار پرنورتر از آبی دیده می‌شود. بنابراین از ضرایب ویژه‌ای برای وزن‌دهی به کانال‌های رنگی و محاسبه روشنایی نهایی تصویر استفاده می‌شود.

```
def rgb_to_gray_luminance(image):  
    image = image / 255  
    red_channel = image[:, :, 0]  
    green_channel = image[:, :, 1]  
    blue_channel = image[:, :, 2]  
  
    final_gray = (0.2126 * red_channel) + (0.7152 * green_channel) + (0.0722 * blue_channel)  
    final_gray = final_gray * 255  
    return final_gray
```

نتایج این روش به شرح زیر هستند:





**روش سوم) تقریب خطی گاما:** چشم انسان در ناحیه روشنایی پایین حساس تر است، بنابراین فشرده سازی گاما توزیع مقادیر رنگ را طوری تغییر می دهد که جزئیات مناطق تیره بهتر حفظ شوند و از نواربندی جلوگیری شود. برای محاسبه دقیق روشنایی باید این فشرده سازی را با گسترش گاما معکوس کرد، اما این کار محاسبات را سنگین می کند. در مواقعی که سرعت مهم تر است، از یک تقریب خطی ساده به جای محاسبات کامل گاما استفاده می شود.

```
def rgb_to_gray_linear_approximation(image):  
    image = image / 255  
    red_channel = image[:, :, 0]  
    green_channel = image[:, :, 1]  
    blue_channel = image[:, :, 2]  
  
    final_gray = (0.299 * red_channel) + (0.587 * green_channel) + (0.114 * blue_channel)  
    final_gray = final_gray * 255  
    return final_gray
```

نتایج این روش به شرح زیر هستند:



## پاسخ پرسش ششم

پس از بررسی این سه روش، اکنون به دنبال روشی هستیم که کمترین میزان خطا را بین تصویر رنگی و تصویر خاکستری ایجاد کند. نتایج هر روش پس از تبدیل به خاکستری در جدول زیر نمایش داده شده است. این مقادیر برای هر سه تصویر آزمایشی ارائه شده‌اند.

| ÷ | Test       | ÷ | Method               | ÷ | Weights                  | ÷ | MSE        | ÷ | PSNR      |
|---|------------|---|----------------------|---|--------------------------|---|------------|---|-----------|
| 0 | Test_0.ppm |   | Average              |   | [0.3333, 0.3333, 0.3333] |   | 406.507205 |   | 22.040121 |
| 1 | Test_1.ppm |   | Average              |   | [0.3333, 0.3333, 0.3333] |   | 515.603866 |   | 21.007642 |
| 2 | Test_2.ppm |   | Average              |   | [0.3333, 0.3333, 0.3333] |   | 632.527341 |   | 20.120011 |
| 3 | Test_0.ppm |   | LUM                  |   | [0.2126, 0.7152, 0.0722] |   | 472.955072 |   | 21.382605 |
| 4 | Test_1.ppm |   | LUM                  |   | [0.2126, 0.7152, 0.0722] |   | 575.935576 |   | 20.527065 |
| 5 | Test_2.ppm |   | LUM                  |   | [0.2126, 0.7152, 0.0722] |   | 746.212540 |   | 19.402178 |
| 6 | Test_0.ppm |   | Linear Approximation |   | [0.299, 0.587, 0.114]    |   | 460.720007 |   | 21.496433 |
| 7 | Test_1.ppm |   | Linear Approximation |   | [0.299, 0.587, 0.114]    |   | 567.399317 |   | 20.591916 |
| 8 | Test_2.ppm |   | Linear Approximation |   | [0.299, 0.587, 0.114]    |   | 707.560469 |   | 19.633168 |

حال پس از نمایش برای هر تصویر، میانگین خطای هر روش را نیز محاسبه کرده‌ایم.

| ÷ | Method               | ÷ | Mean MSE   |
|---|----------------------|---|------------|
| 0 | Average              |   | 461.055536 |
| 1 | LUM                  |   | 524.445324 |
| 2 | Linear Approximation |   | 514.059662 |

همان‌طور که مشاهده می‌شود، روش میانگین‌گیری از هر سه کانال، کمترین میزان MSE را ایجاد می‌کند. علاوه بر نتایج تجربی، این موضوع را می‌توان به صورت **تئوری** نیز اثبات کرد؛ به طوری که استفاده از میانگین ضرایب برای ترکیب کانال‌ها منجر به **کمینه شدن MSE** خواهد شد.

For a single pixel with color vector  $(R, G, B)$ , let the corresponding grayscale value be  $g$ . If we reconstruct the color from gray as  $(g, g, g)$ , the per-pixel squared error is:

$$E(g) = (R - g)^2 + (G - g)^2 + (B - g)^2.$$

To find the  $g$  that minimizes  $E(g)$ , differentiate with respect to  $g$  and set it to zero:

$$\frac{dE}{dg} = -2(R - g) - 2(G - g) - 2(B - g) = 0 \quad \Rightarrow \quad 3g = R + G + B.$$

Thus, the optimal grayscale value for minimizing the per-pixel error is:

$$g = \frac{R + G + B}{3}.$$

Summing over all pixels gives the global minimum of the mean squared error (MSE), since the objective decomposes independently for each pixel.

Computational complexity:  $O(\text{number of pixels})$  arithmetic operations.

## پاسخ پرسش هفتم

در این بخش تابعی پیاده‌سازی کرده‌ایم که برای محاسبه آنتروپی تصویر از آن استفاده می‌کنیم. تابع آنتروپی یک تصویر را با محاسبه توزیع احتمالی مقادیر پیکسل‌ها محاسبه می‌کند. ابتدا تصویر را با توجه به فرمت آن mat یا dcm یا دیگر فرمت‌ها) می‌خواند و به یک آرایه‌ی یک‌بعدی تبدیل می‌کند. سپس تعداد وقوع هر مقدار پیکسل را می‌شمارد و احتمال هر مقدار پیکسل را با تقسیم بر تعداد کل پیکسل‌ها به دست می‌آورد. در نهایت با استفاده از فرمول آنتروپی مجموع حاصلضرب احتمالات در لگاریتم آن‌ها را محاسبه کرده و مقدار آنتروپی تصویر را بازمی‌گرداند.

```
def my_entropy(image_name: str):
    if image_name.lower().endswith(".dcm"):
        ds = pydicom.dcmread(image_name)
        img = ds.pixel_array

    elif image_name.lower().endswith(".mat"):
        mat_data = loadmat(image_name)
        keys = [k for k in mat_data.keys() if not k.startswith("__")]
        img = mat_data[keys[0]]

    else:
        img = cv2.imread(image_name, cv2.IMREAD_UNCHANGED)

    arr = img.reshape(-1)

    counts = np.bincount(arr)
    p = counts[counts > 0] / len(arr)
    entropy = -np.sum(p * np.log2(p))

    return entropy
```

در جدول زیر، آنتروپی هر تصویر را محاسبه کرده‌ایم همچنین زمان اجرای الگوریتم بر هر تصویر را نیز نمایش داده‌ایم.

| ÷  | Sample       | ÷ | Entropy   | ÷ | Runtime  | ÷ |
|----|--------------|---|-----------|---|----------|---|
| 0  | Cells.tif    |   | 7.778102  |   | 0.010995 |   |
| 1  | Pavia.mat    |   | 11.488391 |   | 1.347553 |   |
| 2  | IM000001.dcm |   | 5.632502  |   | 0.010000 |   |
| 3  | IM000002.dcm |   | 5.830014  |   | 0.013000 |   |
| 4  | IM000003.dcm |   | 5.967805  |   | 0.010002 |   |
| 5  | IM000004.dcm |   | 6.172665  |   | 0.008999 |   |
| 6  | IM000005.dcm |   | 6.388312  |   | 0.009000 |   |
| 7  | IM000006.dcm |   | 6.527468  |   | 0.008999 |   |
| 8  | IM000007.dcm |   | 6.506911  |   | 0.009058 |   |
| 9  | IM000008.dcm |   | 6.633951  |   | 0.009801 |   |
| 10 | IM000009.dcm |   | 6.694684  |   | 0.009000 |   |
| 11 | IM000010.dcm |   | 6.711004  |   | 0.009067 |   |
| 12 | IM000011.dcm |   | 6.727748  |   | 0.009007 |   |
| 13 | IM000012.dcm |   | 6.766372  |   | 0.008983 |   |
| 14 | IM000013.dcm |   | 6.797497  |   | 0.009011 |   |
| 15 | IM000014.dcm |   | 6.820334  |   | 0.008012 |   |
| 16 | IM000015.dcm |   | 6.790401  |   | 0.008983 |   |
| 17 | IM000016.dcm |   | 6.829266  |   | 0.008999 |   |
| 18 | IM000017.dcm |   | 6.797693  |   | 0.009013 |   |
| 19 | IM000018.dcm |   | 6.765341  |   | 0.008930 |   |
| 20 | IM000019.dcm |   | 6.783004  |   | 0.008002 |   |
| 21 | IM000020.dcm |   | 6.794988  |   | 0.008261 |   |
| 22 | IM000021.dcm |   | 6.769847  |   | 0.009998 |   |
| 23 | IM000022.dcm |   | 6.763645  |   | 0.009002 |   |
| 24 | IM000023.dcm |   | 6.725397  |   | 0.008997 |   |
| 25 | IM000024.dcm |   | 6.734971  |   | 0.009000 |   |
| 26 | IM000025.dcm |   | 6.721651  |   | 0.009000 |   |
| 27 | IM000026.dcm |   | 6.683261  |   | 0.008995 |   |
| 28 | IM000027.dcm |   | 6.703152  |   | 0.007998 |   |
| 29 | IM000028.dcm |   | 6.673334  |   | 0.009084 |   |
| 30 | IM000029.dcm |   | 6.619660  |   | 0.008984 |   |
| 31 | IM000030.dcm |   | 6.545978  |   | 0.008998 |   |
| 32 | IM000031.dcm |   | 6.448852  |   | 0.008942 |   |
| 33 | IM000032.dcm |   | 6.299636  |   | 0.008991 |   |
| 34 | IM000033.dcm |   | 6.110095  |   | 0.009007 |   |
| 35 | IM000034.dcm |   | 5.907127  |   | 0.008000 |   |
| 36 | IM000035.dcm |   | 5.658393  |   | 0.009000 |   |
| 37 | IM000036.dcm |   | 5.418581  |   | 0.009006 |   |